

High-Performance Computer Architectures

Practical Course

Script

High-Performance Computing

Prof. Dr. Ivan Kisel

Robin Lakos

Akhil Mithran

Oddharak Tyagi

May 6, 2025

Dear students,

Welcome to the High-Performance Computer Architectures practical course. The goal of this course is to provide you an introduction to software development for high-performance computers including simplified applications from our daily research routine. The modern low-level programming language C++ allows to develop software close to the hardware specifications and allows to program runtime- and memory-efficient applications, running on CPU and GPU. Within this course, you will learn how to work with Unix-based systems and C++ to write smaller applications that make use of several concepts of parallel programming such as multi-threading and vectorization.

If you have any questions regarding the examples given, please ask your tutor.

Please note: During the course, we will add new sections and provide more examples for the specific topics to provide a good overview of the course requirements. If you have any suggestions or wishes that could help you, please let us know.

Contents

1	Introduction - Unix Shell and C++	3
1.1	Unix Shell	3
1.1.1	Terminal and Shell Commands	4
1.2	C++ Basics	5
1.2.1	The History and Evolution of C++	5
1.2.2	Learning Resources for C++	5
1.2.3	Repetition: Programming Languages	6
	How Computers Understand Instructions	6
	C++ as a Compiled Language	7
1.2.4	Compiling C++ Source Code - Single Source File	7
	The Minimal C++ Program	7
	The "Hello World!" Program	8
1.2.5	Primitive/Fundamental Data Types in C++	9
1.2.6	Types and Casting	10
1.2.7	Variables, Literals, and Expressions	11
1.2.8	Initialization in C++	12
1.2.9	Pointers in C++	12
1.2.10	References in C++	14
1.2.11	Const Correctness	15
	Const References as Return Types	15
1.2.12	Control Flow	16
1.2.13	Stack vs. Heap	17
1.2.14	Classes and Structs	17
1.2.15	Templates	19
1.2.16	Multi-File Projects and (Meta-)Build-Tools	20

Chapter 1

Introduction - Unix Shell and C++

The Unix Shell and C++ are essential tools for anyone working in high-performance computing or scientific research. Both provide robust capabilities for managing and executing tasks efficiently. This chapter is divided into two sections: the first introduces the Unix Shell as a versatile command-line interface for interacting with operating systems, and the second provides an introduction to C++ programming, emphasizing its role in computational science.

1.1 Unix Shell

The Unix Shell acts as a powerful interface between users and the operating system, allowing direct interaction through text-based commands. Its flexibility and precision make it an indispensable tool for professionals, particularly in high-performance computing, where efficiency and automation are crucial. Unlike graphical user interfaces (GUIs), which prioritize user-friendliness, the shell excels in handling complex, repetitive, and large-scale tasks with ease, thanks to its focus on command-line execution and scripting.

Unix, developed in 1969, laid the foundation for many modern operating systems, including Linux and macOS. Its principles of multi-tasking, multi-user support, and portability have made it a cornerstone of computing systems worldwide. Today, Linux, a Unix-based kernel, powers a vast array of devices, from personal computers to high-performance computing (HPC) clusters and servers.

Linux's open-source nature is one of its greatest strengths in HPC and research. It allows developers and researchers to inspect, modify, and optimize the source code to suit specific needs. This flexibility, coupled with contributions from a global community, ensures robust performance, rapid issue resolution, and high adaptability for demanding computational tasks.

Linux distributions come in many flavors, tailored to different use cases. For instance, Debian and Fedora are often chosen for servers due to their stability and reliability. Conversely, distributions like Ubuntu and Arch Linux cater to desktop users with user-friendly interfaces and up-to-date software. The availability of diverse Desktop Environments (DEs) further allows users to customize their workflows, enhancing productivity and accessibility.

In the realm of HPC, Linux dominates as the operating system of choice. Its stability, scalability, and extensive support for open-source tools and frameworks make it indispensable for running large-scale simulations, managing big data, and optimizing computational workflows.

1.1.1 Terminal and Shell Commands

In this course, all C++ applications will be compiled and executed via the terminal (also referred to as the console), using the Unix Shell. Working through the terminal allows precise control and avoids the overhead introduced by graphical user interfaces or additional software. While initially it may feel less intuitive, the terminal is a powerful tool for efficient computing, particularly in high-performance computing environments.

When you start a terminal in Linux, the prompt typically displays `username@hostname`, followed by a *path*, which represents the current *working directory*. By default, the working directory is usually set to the user's home directory. Note that terminal prompts and styles may vary depending on the application or theme being used.

Key Commands for Navigation

The following table summarizes essential commands for navigating the file system in the Unix Shell:

Command	Description
<code>pwd</code>	Prints the working directory, showing the absolute path to the current location.
<code>cd</code>	Changes directory. Paths can be absolute (starting from the root directory, <code>/</code>) or relative (from the current directory).
<code>ls</code>	Lists the contents of the current directory.

Table 1.1: Essential Unix Shell commands for navigation.

Below is an example of using these commands in a typical workflow:

```

1  username@hostname:~$ pwd
2  /home/hpc
3  username@hostname:~$ ls
4  directory-A directory-B directory-C
5  username@hostname:~$ cd directory-A
6  username@hostname:~/directory-A$ ls
7  directory-1 directory-2
8  username@hostname:~/directory-A$ cd directory-1/directory-1-1
9  username@hostname:~/directory-A/directory-1/directory-1-1$ cd ../../..
10 username@hostname:~$

```

Switching between directories with long paths can become tedious. The command `cd -` offers a convenient shortcut, allowing you to quickly switch back to the last directory you were working in. This is especially helpful when toggling between two separate locations:

```

1  username@hostname:~$ pwd
2  /home/hpc
3  username@hostname:~$ cd directoryA/directory-1/directory-1-1
4  username@hostname:~/directoryA/directory-1/directory-1-1$ DO SOMETHING HERE
5  username@hostname:~/directoryA/directory-1/directory-1-1$ cd
   ↪  ../../../../directoryB/directory-3/directory-2-1

```

```
6 username@hostname:~/directoryB/directory-3/directory-2-1$ DO SOMETHING THERE
7 username@hostname:~/directoryB/directory-3/directory-2-1$ cd -
8 username@hostname:~/directoryA/directory-1/directory-1-1$ DO SOMETHING HERE
  ↪ AGAIN
```

Mastering the shell is essential for anyone aiming to work efficiently in high-performance computing. Throughout this course, make an effort to familiarize yourself with the terminal, as it will be a key tool for compiling, running, and managing your C++ applications.

1.2 C++ Basics

C++ is one of the most widely used programming languages in scientific computing and high-performance applications. Known for its efficiency, flexibility, and extensive control over system resources, C++ combines the power of low-level hardware access with modern high-level abstractions. These features make it an indispensable tool in fields such as computational physics, engineering simulations, and data-intensive research. In this section, we introduce the foundational concepts of C++ and its relevance to modern programming.

1.2.1 The History and Evolution of C++

C++ was first developed by Bjarne Stroustrup in 1985 as an extension of the C language. The goal was to combine the performance and portability of C with features supporting object-oriented programming, such as classes and objects. This key enhancement distinguished C++ as one of the first mainstream programming languages to support object-oriented design, alongside procedural and generic programming paradigms. C++ has grown significantly since its inception, evolving through multiple standards established by the International Organization for Standardization (ISO). Starting from C++98, these standards have introduced features like templates, smart pointers, and lambda expressions, with major updates occurring every three years since 2011. This makes C++ a "modern" language that balances its low-level control with high-level abstractions. While C++ is often considered a high-level language due to its expressive syntax, it retains low-level features like direct memory management and hardware-level operations. This versatility positions C++ between low-level languages like Assembly and modern high-level languages like Python, making it highly efficient and performant, particularly for computationally intensive tasks.

1.2.2 Learning Resources for C++

Given the breadth of the language, this course will focus on essential concepts and practices relevant to high-performance computing. For more comprehensive learning, the following resources are highly recommended:

Online resources:

- <https://cppreference.com/>
A precise and extensive reference for C++ language features and libraries, complete with examples.
- <https://isocpp.github.io/CppCoreGuidelines/>
A guide to writing efficient, maintainable, and safe C++ code.

1.2.3 Repetition: Programming Languages

Before diving into C++, this section revisits some foundational concepts of programming to ensure a clear understanding of key terms and ideas.

Definitions:

- A *computer* is a device that executes programs.
- A *program* is a precise sequence of instructions, enabling a computer to perform a specific task.

How Computers Understand Instructions

At the core of programming is the interaction between a computer and the instructions it executes. To fully appreciate the role of programming languages like C++, it's crucial to understand how computers interpret and process instructions at the most fundamental level.

A computer fundamentally "understands" binary, as its electronic components operate based on two states: voltage present (1) or not present (0). This binary logic is foundational to digital computing, where thresholds are applied to distinguish between these states, ensuring resilience against fluctuations and noise. Boolean logic is then used to process these signals efficiently.

The exact binary instructions a computer understands depend on its hardware architecture. For instance, ARM-based processors interpret instructions differently than x86_64-based processors. Data is represented in binary, too, but its interpretation depends on the specified *data type*. Consider the following binary sequence:

01100101 01110110 01101001 01101100

- interpreted as a 32-bit float (IEEE-754) results in 7.272793e22.
- interpreted as a 32-bit integer results in 1702259052.
- interpreted as ASCII/UTF-8 text results in "evil".

Understanding how computers work with binary at this level forms the foundation for learning how programming languages bridge the gap between human-readable instructions and machine-readable operations.

While computers operate on binary instructions, it would be impractical and time-consuming for humans to write programs directly in binary. High-level programming languages were developed to simplify this process, allowing humans to write instructions that are closer to natural language while maintaining the precision required for computers.

High-level programming languages provide an abstraction layer that shields programmers from the complexity of binary or low-level machine instructions. Unlike human languages, which allow for interpretative flexibility, programming languages are strictly defined with precise syntax

and semantics. Each statement corresponds to a specific operation defined by the language's rules.

Since computers only "speak" binary, instructions written in high-level languages need to be translated into machine-readable binary instructions. This translation is typically performed by a *compiler* or an *interpreter*:

- A *compiler* translates the entire source code into a binary executable file at once. This executable can then be run repeatedly without needing the source code, as long as the target computer uses the same architecture.
- An *interpreter*, on the other hand, translates and executes source code line-by-line at runtime, without producing a standalone executable.

This distinction between compilers and interpreters highlights a key difference in how programming languages are implemented, which is particularly relevant when considering C++.

C++ as a Compiled Language

C++ is a high-level programming language that is almost¹ always implemented as a compiled language. This means that programs written in C++ are first translated into binary executable files by a compiler before they can be executed.

This compilation process gives C++ its reputation for efficiency and speed, as the resulting binary is optimized for the specific architecture on which it will run. Numerous vendors provide C++ compilers, including the Free Software Foundation, LLVM, Microsoft, Intel, Embarcadero, Oracle, and IBM. For this course, the recommended compilers are:

- g++ (GNU Compiler Collection) by the Free Software Foundation.
- clang++ by LLVM.

By understanding the role of compilers and how they relate to C++, you'll be better equipped to write efficient programs and manage their execution across different systems.

1.2.4 Compiling C++ Source Code - Single Source File

As already mentioned, the compilation of source code is the translation from a programming language into machine instructions. For a single file, this process is more intuitive, so we begin with a simple example. Later, we will explore more complex projects in greater detail.

The Minimal C++ Program

Before diving into a full-fledged example, let's create the simplest possible C++ program, containing only an empty `main()` function. This serves as a starting point to demonstrate how to compile and execute C++ code.

Create a plain text file with the `.cpp` extension, named `Minimal.cpp`, and add the following content:

¹Some Just-In-Time (JIT) compilers, such as the one included in the ROOT Framework developed at CERN, enable C++ to behave somewhat like an interpreted language.

```
1 int main()
2 {
3     return 0;
4 }
```

Explanation:

- The `main()` function is the entry point of every C++ program. When the program is executed, this function is called first.
- The `return 0;` statement signals to the operating system that the program has executed successfully. Returning a non-zero value typically indicates an error.

To compile and execute this minimal program:

1. Compile the source code into an executable file using the `clang++` compiler:
`clang++ Minimal.cpp -o Minimal.out`
2. Run the resulting executable:
`./Minimal.out`
3. Since this program has no output, nothing will appear in the console.

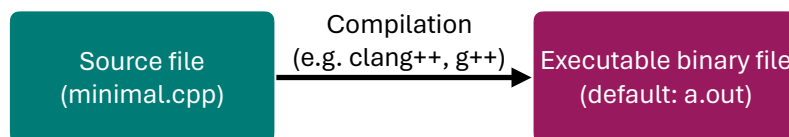


Figure 1.1: Translation of C++ source code into executable machine instructions.

This minimal example demonstrates the steps to compile and run a C++ program.

The "Hello World!" Program

Now, let's expand on this by creating a more illustrative program. The "Hello World!" example introduces basic input and output functionality in C++. Start by creating another file, named `HelloWorld.cpp`, and add the following content:

```
1 #include <iostream>
2
3 int main()
4 {
5     std::cout << "Hello world!" << std::endl;
6     return 0;
7 }
```

Explanation:

- The `#include <iostream>` directive imports the `iostream` library, enabling input and output operations.

- `std::cout` is used to output text to the console. The `<<` operator streams the string "Hello world!" to the standard output.
- `std::endl` adds a newline character and flushes the output buffer, ensuring the output appears immediately.
- The program ends with `return 0;`, signaling successful execution.

To compile and execute this program:

1. Compile the source code:
`clang++ HelloWorld.cpp -o HelloWorld.out`
2. Run the executable:
`./HelloWorld.out`

These two examples - `Minimal.cpp` and `HelloWorld.cpp` - provide a foundational understanding of compiling and running C++ programs.

1.2.5 Primitive/Fundamental Data Types in C++

There are several *primitive* or *fundamental data types* in C++, such as those listed in [Table 1.2](#), that help programmers to build more complex structures and data types.

Data Type	(Typical) Size in Bits
<code>bool</code>	1
<code>char</code>	8
<code>int</code>	32
<code>long</code>	64
<code>float</code>	32
<code>double</code>	64

Table 1.2: Example of a few fundamental data types and their typical sizes on modern PCs.

Here, these data types are given with their typical sizes in bytes. In the C++ standard, sizes of fundamental data types are given as minimum required precision, not as a fixed size value. For example, the data type `int` requires at least 16 Bit / 2 Bytes, but usually comes with 32 Bit / 4 Bytes on modern computer architectures. More on that here: <https://en.cppreference.com/w/cpp/language/types>.

This decision is rooted historically and is influenced by the need for portability and compatibility (e.g. backward compatibility) across various platforms, including those with different word sizes and memory constraints, e.g. in embedded systems.

These sizes are sometimes neglected as long as the used data types are sufficiently large to store the data. In high-performance computing, however, these can have crucial factors with respect to the runtime. For example, the charge of a particle in heavy-ion physics experiments can have one of three different states: positively (+1), neutrally (0) or negatively (-1) charged. Therefore, a datatype that can store at least three different states is sufficient.

A `bool` with only two states (true, false) is not sufficient and the next largest integer type that can represent it is a `short` with 16 Bit on modern computers. Intuitively, this would make sense as it

can easily store negative and positive numbers. However, a `char` has a size of 8 Bits and can also easily represent the three states. Although it is a character type, it can be used to store the charge of a particle. For each particle, there are 2 times less Bits required to store its electrical charge when using `char` in comparison to `short`.

But does it make a difference? Well, it depends. If the computer reads a line of aligned memory (RAM) and stores it into the cache, twice as much values can be stored when we would store the charges of particles as `char`. Access to the memory is quite costly, that sometimes even re-calculation can be much faster than reading it from memory. Just to provide some numbers: a memory access can easily take up to 100 ns, whereas L1 cache access can be as fast as 0.5 ns. Nevertheless, to achieve an efficient runtime, optimal data types help definitely, but how they are stored and aligned in memory is at least as crucial as finding the best matching one. Later in this course, different types of storing data are discussed in more detail.

1.2.6 Types and Casting

C++ is a statically and strongly typed language, which means the types of variables must be known at compile-time. This attribute enforces type safety by generating errors if variable types do not match during compilation, limiting implicit conversions. It is imperative in C++ that every variable and function return type is explicitly declared. In C++ 11, the *auto* keyword was introduced, enabling the compiler to infer variable types at compile time, enhancing code readability and maintainability, especially in complex scenarios like iterating over containers or using lambdas.

Regarding type casting, C++ supports conversions between different types, categorized into implicit and explicit conversions. Implicit conversions are automatically performed by the compiler, whereas explicit conversions require specific syntax to instruct the compiler. Although C++ maintains backward compatibility with C-style casting, it is advisable to utilize C++'s four cast operators for clearer, type-safe conversions. This recommendation stems from the fact that C-style casting can allow conversions between incompatible types, posing risks for code maintenance and clarity, especially in collaborative environments.

The C++ cast operators are:

- `static_cast<T>`: Suitable for non-polymorphic type conversions, including upcasting and downcasting within class hierarchies, numeric type conversions, and pointer type conversions where implicit or reversible. This operator ensures type safety through compile-time checks.
- `dynamic_cast<T>`: Utilized for safe downcasting in class hierarchies, converting a base class pointer or reference to a derived class pointer or reference. It employs Run-Time Type Information (RTTI) to verify the cast's validity, returning a null pointer or throwing a `bad_cast` exception upon failure.
- `reinterpret_cast<T>`: Enables conversions between pointer types or between pointer and integer types, performing a bit-wise copy of the value. Its use is generally discouraged due to potential alignment and type punning issues, and should only be considered when absolutely necessary.

- `const_cast<T>`: The only cast capable of modifying the constness of an object, useful for invoking non-const methods on const objects or modifying objects initially declared as const. This operator can add or remove the `const`, `volatile`, or `__unaligned` qualifier from a pointer or reference.

Transitioning from the basics of types and casting, let's delve into practical examples to illustrate these concepts further.

Examples:

```

1  // different static type-casting variants
2  short a = 10;
3  float b;
4  b = a;                      // implicit (variant 0)
5  b = (float) a;              // explicit (variant 1)
6  b = float(a);               // explicit (variant 2)
7  b = static_cast<float>(a);  // explicit (variant 3)
8
9  // dynamic type-casting for pointers/references to objects
10 Animal* animal = new Dog();
11 Dog* dog = dynamic_cast<Dog*>(animal);
12
13 // reinterpreting memory, here array interpreted as __m128 vector:
14 float data[] = {1.f, 2.f, 3.f, 4.f};
15 __m128 &vec = reinterpret_cast<__m128&>(data);
16
17 // modifying constness of an object to re-write const-objects:
18 int c = 10;
19 const int& d = c;             // create "write-protected" reference
20 const_cast<int&>(d) = 5;      // re-write "write-protected" reference

```

1.2.7 Variables, Literals, and Expressions

In C++, *variables* are identifiers providing a name to objects in memory, for instance, `int a;` defines a variable named `a` of type `int`. *Literals* represent constant values directly included in the code, like `5` in `a = 5;`. An *expression* involves operators and operands performing computations, such as `a++`, which increments `a`.

Each object in C++ is characterized not just by its data type and size, but also by two main properties:

- An address, indicating the memory location of the object.
- Content, the data value stored at the object's memory location.

```

1  int a = 42; // Creates an integer variable named 'a' with a value of 42
2  a = 1337;  // Modifies the content of 'a' to 1337

```

1.2.8 Initialization in C++

C++ supports various initialization methods. Traditional forms include:

- Direct Initialization: `int a(5);`
- Copy Initialization: `int a = 5;`

C++11 introduced *Brace Initialization*, offering advantages such as a uniform syntax across all types, prevention of narrowing conversions, and uniform initialization of aggregates (arrays, structs):

```
1  int a{4};      // No error
2  int b{7.9};    // Compiler error due to narrowing
```

Initialization of data structures:

```
1  std::vector<int> v{1, 2, 3};
2  int array[]{1, 2, 3};
3  YourClass obj{};
```

This initialization method enhances code safety and is thus recommended in modern C++ practices.

1.2.9 Pointers in C++

Your introduction effectively sets the stage for understanding memory addresses. Clarifying that every memory cell can be addressed and that variables are allocated to these addresses could be beneficial. It might be helpful to mention that the size and alignment of variables affect how they are placed in memory, which is managed by the compiler and the runtime environment.

Any variables in C++ can be accessed with its identifier (name). This is the easiest way when we don't need to care about the physical location of our data in the memory. However, there is a second way to access a variable, directly using its location in the memory. The computer's memory can be represented as a sequence of memory cells of one byte each. These cells are numbered in a consecutive way. Each cell has a unique number in the whole available memory. Every next cell has the number of the previous one plus one. This way we can claim that the cell number 55 definitely follows the cell number 54. As soon as we declare a variable, the amount it needs in memory is allocated in a specific location (memory address). The operating system performs this task automatically during runtime. This memory address locates a variable in the memory. This memory address can be obtained by adding an ampersand sign (&), the so-called *address operator*, in front of the identifier. So &a gets the address of the variable a. A variable which stores the memory address of another variable is called a pointer.

Pointers are a foundational aspect of C++, allowing variables to reference memory locations of objects. Key points regarding pointers include:

- Pointers have a consistent size, typically making them more efficient to move or copy compared to large data structures.

- Modifying an object via a pointer affects the original object globally.
- Pointers facilitate data structure manipulations, such as swapping nodes in trees without moving all child nodes.

Pointers are said to "point to" the variable whose address they store. All pointers have a special pointer type. While declaring a pointer, one has to specify this type. The pointer type is obtained by adding an asterisk after the type it points to.

```
1 int a = 5;      // 5 stored at some memory location, let's say at 0x7ffcdf8a990c
2 int* b = &a;    // 'b' points to the address of 'a' (0x7ffcdf8a990c)
3                // 'b' itself is located somewhere else, e.g. at 0x7ffcdf8a9900
```

The content of a pointer is a memory address and the address where the pointer is pointing to has some content, which can be accessed by dereferencing the pointer.

Dereferencing Pointers: The value (content) at the pointer's referenced memory location can be accessed by dereferencing the pointer. One can use the asterisk sign (*) in front of the pointer variable to dereference the pointer. In this case, the * is called *dereference operator*.

```
1 std::cout << "address of a: " << &a << ", content of b: " << b << std::endl;
2 std::cout << "content of a: " << a << ", dereferenced b: " << *b << std::endl;
3 std::cout << "address of b: " << &b << std::endl;
```

Additionally, operations can be applied to pointers, like arithmetic operations to traverse arrays.

```
1 int arr[2];
2 int* p = arr; // same as "int* p = &arr[0]" -> p points to the first element
3 ++p;         // p points now to the second element
```

In this example, the behavior of `arr` is similar to pointer: one can either set `p` to `arr` or to the address of element 0 of the array, but not to the address of `arr`. It might seem like the content of `arr` is an address, similar to pointers, but actually `arr` is a array-type. But arrays in expressions decay to pointers to their first element, reflecting a design choice from C for simplicity and efficiency.

This conversion facilitates passing arrays to functions as pointers, enabling efficient data manipulation. However, a critical implication of array decay is the loss of size information. When an array decays to a pointer, the function receiving the pointer does not inherently know the size of the original array. This necessitates careful management, often requiring the size of the array to be passed as an additional parameter to functions that operate on the array. Understanding and accounting for array decay is essential to avoid out-of-bounds access, ensuring safe and effective use of arrays in C++. In modern C++, other container classes/structs are usually used, such as `std::vector<T>` or `std::array<T>`.

Fundamentally, a pointer is designed to hold the address of a single memory location. However, when it references an object occupying more than one byte in memory – such as a multi-byte

data type or an array – the pointer is actually pointing to the initial byte of a contiguous block of memory where the object resides. For instance, a pointer referencing a 4-byte integer, or an array thereof, essentially points to the first byte of this integer. When such a pointer is incremented, it advances to the next element in the sequence. The magnitude of this advancement, or "jump", is determined by the size of the object type to which the pointer refers. Consequently, for an array of 4-byte integers, incrementing the pointer results in it moving 4 bytes forward, aligning with the start of the subsequent element in the array. Thus, pointer-arithmetic has type-awareness.

Please note: The `this` keyword in C++, representing the object itself, is also a pointer.

1.2.10 References in C++

References in C++ create an alias for an existing variable, serving as a powerful tool in high-performance computing by circumventing the need for costly copies. They find their primary utility in two scenarios: as return types and as function parameters, significantly enhancing performance by operating directly on the original objects.

Consider the distinction between working with a copy versus a reference:

Working with a copy:

```
1 Matrix matrix = obj.GetLargeMatrix(); // Incurs significant overhead by copying
2 matrix.DoSomething();
```

Working with a reference:

```
1 Matrix& matrix = obj.GetLargeMatrix(); // Efficiently accesses the original
2 matrix.DoSomething();
```

To utilize a reference, one has to change the function's return type to return a specific reference type, e.g. `Matrix& GetLargeMatrix()`.

This technique is similarly applied to function parameters to ensure that operations are performed on the original data without incurring the overhead of copying. Function parameters declared with an `&`-sign are treated as references, enabling direct manipulation of arguments as if they were the originals. This approach is not only efficient but also maintains code clarity and intuitive logic flow. However, it is crucial to ensure that returned references do not point to local objects within a function scope to avoid dangling references and the errors resulting from this.

```
1 Matrix MatrixMultiplication(const Matrix& a, const Matrix& b)
```

Here, two references of type `Matrix` are expected as function parameters. In a non-static function that is applied directly to an object, one matrix (e.g. `Matrix 'a'`) could be the object the function is applied to and then only one matrix (e.g. `Matrix 'b'`) would be given as a parameter.

However, this is somewhat a design choice. In some cases, in high-performance computing, you will also see that the result reference is given as a function parameter. For example, when memory is already pre-allocated for re-using it multiple times, such a concept makes sense and

avoids multiple re-allocations of large spaces in memory. In the end, this is usually a trade-off between intuitive code and performance. For maintenance reasons, you should not always opt for performance, but in limited environments, it might help.

The return type here is not a reference, as in this example, it is assumed that the result is a new matrix and not one of the two operands is overwritten. When creating objects inside a function body that is then not stored in another object with a longer lifetime, one can not return a reference to that object. When the function body (scope) ends, the lifetime of the new object ends and the reference would be referring to an object that does not exist anymore, leading to an error.

Additionally, in the example above, the `const` keyword is used to avoid accidental modifications to the matrices. The keyword will be discussed in more detail later on.

A reference is implemented similarly to a pointer (e.g. it has a small size of 8 bytes in 64-Bit systems), while a copy is always depending on the target object's size.

When working with large data structures, copying a large object is timely expensive and should be avoided whenever possible.

1. Memory has to be allocated for the new object
2. The whole object has to be copied into the new location

Avoiding unnecessary copies not only saves time but also reduces memory usage, which is critical in high-performance computing and memory-constrained environments like embedded systems.

1.2.11 Const Correctness

Ensuring const correctness in function parameters and return types is a best practice in C++ programming. This involves using `const` with references to prevent modification of the original object, enhancing code safety and expressiveness.

Example:

```
1 void DisplayMatrix(const Matrix& matrix) {  
2     // Display the matrix without modifying it  
3 }
```

Applying `const` to reference parameters enforces immutability, allowing read-only access to the object. This guarantees that the function will not alter the object's state, providing clarity to other developers and preventing unintended side effects.

Furthermore, const correctness aids in the optimization of compiler-generated code, potentially unlocking performance enhancements through the assurance that certain objects remain unchanged throughout their use. This can also have a non-negligible effect on the runtime.

Const References as Return Types

While returning a reference can significantly optimize performance by avoiding unnecessary copies, combining it with `const` ensures the returned object is protected against unintended modifications. This technique is particularly useful when returning objects that are members of a

class or have a static lifetime, ensuring the safety and integrity of the returned reference.

Example:

```
1  const Matrix& GetReadOnlyMatrix() {  
2      static Matrix matrix;  
3      // Perform operations  
4      return matrix; // Returns a read-only reference  
5  }
```

Incorporating const correctness into your C++ programming practices promotes not only efficiency and resource conservation but also contributes to the development of robust, error-resistant code.

1.2.12 Control Flow

C++ allows the programmer to have different paths of execution through the program. This is achieved via a collection of control flow statements. The simplest example is perhaps the if-statement that lets us execute a statement only if a conditional expression is true. For ease of understanding we will divide control flow statements into six categories: conditional statements, jumps, function calls, loops, halts and exceptions.

Conditional statements cause a block of code to be executed if and only if a certain condition, such as an equality (==) or inequality (>, <, <= etc.), is satisfied. The if-else and switch statements fall into this category. Jump statements tell the CPU to execute statements at some other location in the program. The break, continue and goto statements are jump statements. Loops tell the program to repeatedly execute some sequence until some condition is met. The while, do-while, for and ranged-for statements are used to generate loops. Exceptions are a special kind of statements designed for error handling such as unexpected user input. C++ uses try, throw and catch for exception handling. You will not encounter many uses of exceptions in this course, but the exception handling is important generally. Function calls are jumps to some other location in the code and back. Function calls and return are used together to achieve this effect. Finally, In some situations the only recourse is to terminate the program. Halt statements like std::exit() and std::abort() statements can be used for this.

C++ allows the programmer to nest control flow statements i.e. it is allowed to use, for example, loops inside loops which may themselves be inside of an if statement. This nesting of control flow statements allows the program to take highly convoluted and input dependent paths. A good command over control flow statements is absolutely necessary to follow the rest of this course. Please spend time to master these statements.

The following example provides at least some applications of conditional statements and loops.

```
1  int a = 5;  
2  int b = 10;  
3  int c = (a > b) ? b : a--;  
4
```

```

5  for (int i = 0; i < c; i++) {
6      do {
7          std::cout << "a: " << a << std::endl;
8          a--;
9      }
10     while (a > -1);
11
12     std::cout << "i: " << i << std::endl;
13 }

```

1.2.13 Stack vs. Heap

There are primarily two types of memory allocations in C++: stack and heap allocations. For stack allocation, the allocation happens on contiguous blocks of memory. We call it a stack memory allocation because the allocation happens in the function call stack. The size of memory to be allocated is known to the compiler and whenever a function is called, its variables get memory allocated on the stack. And whenever the function call is over, the memory for the variables is deallocated. A programmer does not have to worry about memory allocation and deallocation of stack variables. In the case of heap, the memory is allocated during the execution of instructions written by programmers. Every time when we make an object using the operators for dynamic allocation, it is always created in heap-space and the referencing information to these objects is always stored in stack-memory. Heap memory allocation is not as safe as Stack memory allocation because the data stored in this space is accessible or visible to all threads. If a programmer does not handle this memory well, a memory leak can happen in the program.

```

1  int a = 5;           // int allocated on stack
2  int* b = new int(3); // int allocated on heap
3
4  // print out "5 3":
5  std::cout << a << " " << *b << std::endl;
6
7  delete b;           // free heap-allocated memory manually

```

1.2.14 Classes and Structs

Classes and structs are the constructs whereby the user can define their own types. The two constructs are identical in C++ except that in structs the default accessibility is public, whereas in classes the default is private. The term accessibility here refers to the allowed access to the members of the class/struct. Access controls enable you to separate the public interface of a class, which can be interacted with “directly” after an object from the class is created, from the private implementation details, which are set during object creation and changed using member functions of the class, and the protected members that are only for use by derived classes. The access specifier applies to all members declared after it until the next access specifier is encountered. Classes and structs can both contain data members and member functions, which enable you

to describe the type's state and behavior. Structs, however, are often used to create small data structures, whereas classes are often used for more complex objects.

```
1  struct Node {
2      int id;
3      int value;
4  };
5
6  struct Edge {
7      Node* u;
8      Node* v;
9  };
10
11
12  class Graph {
13  private:
14      int nNodes_;
15      int nEdges_;
16
17      Node* nodes_;
18      Edge* edges_;
19  protected:
20      // member functions and variables
21  public:
22      Graph(Node* nodes, Edge* edges, int nNodes, int nEdges) :
23          nodes_(nodes),
24          edges_(edges),
25          nNodes_(nNodes),
26          nEdges_(nEdges)
27      {
28          // other constructor code
29      }
30
31      // member functions and variables
32  };
```

When working with classes in multiple header and source files, it is important to set guards to ensure that every class is included once. This can be done by the following guards:

```
1  #ifndef CLASSNAME_H
2  #define CLASSNAME_H
3
4  class CLASSNAME
5  {
6      // class code here
7  };
```

```

8
9  #endif // CLASSNAME_H

```

1.2.15 Templates

The template system is designed to simplify the process of creating functions (or classes) that are able to work with different data types. Instead of manually writing a bunch of almost identical functions or classes (one for each set of different types), we can instead create a single template. Just like a normal definition, a template describes what a function or class looks like. However unlike a normal definition (where all types must be specified), a template uses one or more placeholder types. A placeholder type represents some type that is not known at the time the template is written, but that will be provided later. Once a template is defined, the compiler can use the template to generate as many overloaded functions (or classes) as needed, each using different actual types! Thus we only have to create and maintain a single template, and the compiler does all the hard work by replacing the types respectively. In fact, the C++ standard library itself is absolutely full of template code.

To create a template function, replace instances of specific types (such as `int` or `float`) with type template parameters which we name, for instance, `T`. Now we have to declare the template parameter to let the compiler know that the function is a template. We do this by adding the following line before the function declaration: `template <typename T>`.

```

1  // function template
2  template <typename T>
3  T GetMax (T a, T b) {
4      if (a > b) return a;
5      return b;
6  }
7
8  // class template
9  template <class T>
10 class Pair {
11 private:
12     T a, b;
13
14 public:
15     Pair (T first, T second) : a(first), b(second)
16     {
17         // other constructor code
18     }
19
20     T GetMax()
21     {
22         if (a > b) return a;
23         return b;

```

```

24     }
25 }

```

1.2.16 Multi-File Projects and (Meta-)Build-Tools

Many projects consist of more than a single source code file. There are several tools that can be used to handle projects. A well-known tool is CMake, that allows to create a configuration file with all relevant information, such as files to compile, compiler settings, dependency checks and much more. In the following example, we assume a project in the `project` directory. In this directory, there is the CMake configuration `CMakeList.txt` that contains all relevant information required for compiling the application. Depending on the project structure, there are code files or other directories with the code included that are then loaded by the `CMakeList.txt`.

To compile our project, a `build` directory is created using the "make directory"-command `mkdir`. A second step is to navigate into this directory using "change directory"-command `cd`. Then we use the CMake tool to setup our code for compilation, using the `CMakeList.txt` in the directory outside of the build directory - CMake will automatically search for it. After this, we can run `make` to build our project and run our application.

```

1  username@hostname:~/project$ mkdir build
2  username@hostname:~/project$ cd build
3  username@hostname:~/project/build$ cmake ../.
4  username@hostname:~/project/build$ make
5  username@hostname:~/project/build$ ./PROJECTNAME

```

A simple `CMakeLists.txt` file could look like this:

```

1  cmake_minimum_required(VERSION 3.22)
2  project(PROJECTNAME)
3
4  set(CMAKE_CXX_STANDARD 14)
5
6  include_directories(.)
7
8  add_executable(PROJECTNAME
9      Project.cpp
10     ClassA.h
11     ClassA.cpp
12     ClassB.h
13     ClassC.cpp)

```